Software Requirements Specification: NeuroShell Intent Recognition Engine (IRE)

NeuroShell Core Team Cyber Operations Research Group
MSc Cybersecurity Research Project R26-CS-018

Abstract

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the NeuroShell Intent Recognition Engine (IRE), designated Component 01 of the NeuroShell Framework. The IRE is a domain-specific Natural Language Understanding (NLU) gateway that converts natural-language offensive-security operator commands into validated, schema-constrained JSON intent contracts. This document specifies the seven-stage pipeline architecture with its eleven middleware components, the hallucination and adversarial detection mechanisms, role-based access control, external interfaces, non-functional requirements, data requirements, and the empirical verification results obtained from a 900-record evaluation campaign.

Index Terms

Software Requirements Specification, Intent Recognition, Natural Language Understanding, Large Language Models, FastAPI, Cybersecurity Automation, Penetration Testing.

Document Control

Document ID	NS-IRE-SRS-001
Version	1.0
Date	2026-09-01
Project	NeuroShell Framework — Component 01
Reference	R26-CS-018 (MSc Cybersecurity Research)
Classification	Internal / Academic Research
Status	Current

Revision History

Version	Date	Author	Changes
1.0	2026-09-01	NeuroShell IRE	Initial comprehensive SRS

Contents

I Introduction 5
I-A Purpose . . . 5
I-B Scope . . . 5
I-B1 Product Scope . . . 5
I-B2 In Scope . . . 5
I-B3 Out of Scope . . . 5
I-C Definitions, Acronyms, Abbreviations . . . 5
I-D References . . . 6

References 6
I-E Overview . . . 6

II Overall Description 6
II-A Product Perspective . . . 6
II-B Product Functions . . . 6
II-C User Classes and Characteristics . . . 7
II-D Operating Environment . . . 7
II-D1 Hardware . . . 7
II-D2 Software . . . 7
II-D3 LLM Models . . . 7

II-E Design and Implementation Constraints . . . 7
II-F Assumptions and Dependencies . . . 8
II-F1 Assumptions . . . 8
II-F2 Dependencies . . . 8

III System Features and Functional Requirements 8
III-A Pipeline Architecture . . . 8
III-A1 Overview . . . 8
III-A2 Stage Sequence . . . 8
III-A3 Middleware Sequence . . . 8
III-B Pre-Inference Middleware (M1–M4) . . . 9
III-B1 M1 — AdversarialDetector . . . 9
III-B2 M2 — RBACGuard (Pre-Inference) . . . 10
III-B3 M3 — SessionContextStore . . . 10
III-B4 M4 — SemanticCache . . . 10
III-C Core Pipeline Stages (S1–S7) . . . 10
III-C1 S1 — InputNormalizer . . . 10
III-C2 S2 — AliasResolver . . . 10
III-C3 S3 — OllamaInferenceEngine (Tier-1/Tier-2) . . . 10
III-C4 S4 — JSONParser . . . 11
III-C5 S5 — SchemaValidator . . . 11
III-C6 S6 — RegexValidator . . . 11
III-C7 S6B — NetworkArchitectureValidator . . . 11
III-C8 S7 — ScopeGuard . . . 12
III-D Post-Inference Middleware (M5–M11) . . . 12
III-D1 M5 — RBACGuard (Post-Inference) . . . 12
III-D2 M7 — SemanticCache (Store) . . . 12
III-D3 M8 — IREResponseV2 Assembly . . . 12
III-D4 M9 — SubIntentClassifier . . . 12
III-D5 M10/M11 — Session Update . . . 12
III-E Schema and Contract System . . . 12
III-E1 IntentSchema (IRE Internal) . . . 12
III-E2 PlannerContract (Component 02 Hand-off) . . . 13
III-F Hallucination Taxonomy and Enforcement . . . 13
III-F1 HallucinationClass Enum (6 Classes) . . . 13
III-F2 Enforcement Policy . . . 13
III-F3 ValidationFinding Schema . . . 13
III-G Contract Builder and Tool Synthesis . . . 13
III-G1 Tool Synthesis Table . . . 13
III-G2 Priority Ladder . . . 13
III-G3 Nmap Flag Mapping . . . 13
III-H Audit and Escalation System . . . 13
III-H1 Audit Logger . . . 13
III-H2 Escalation Queue . . . 13

IV External Interface Requirements 14
IV-A User Interfaces . . . 14
IV-B Hardware Interfaces . . . 14
IV-C Software Interfaces . . . 14
IV-D Communication Interfaces . . . 14
IV-D1 REST API Base . . . 14
IV-D2 Response Headers . . . 14
IV-E API Endpoint Specifications . . . 14
IV-E1 GET / . . . 14
IV-E2 GET /health . . . 14
IV-E3 POST /parse . . . 15
IV-E4 POST /parse/plan . . . 16
IV-E5 POST /execute . . . 16

IV-E6 GET /metrics . . . 16
IV-E7 GET/DELETE /session/{session_id} . . . 16
IV-E8 Admin Endpoints . . . 16

V Non-Functional Requirements 16
V-A Performance Requirements . . . 16
V-A1 Latency (Measured on CPU-only Hardware) . . . 16
V-A2 Throughput . . . 16
V-B Safety Requirements . . . 17
V-C Security Requirements . . . 17
V-C1 Input Security . . . 17
V-C2 Data Security . . . 17
V-C3 Injection Defense Layers . . . 17
V-D Reliability and Availability . . . 17
V-E Maintainability . . . 18
V-F Portability . . . 18

VI Data Requirements 18
VI-A Logical Data Model . . . 18
VI-A1 Core Entities . . . 18
VI-A2 ValidationFinding . . . 18
VI-A3 CacheEntry (Semantic Cache) . . . 19
VI-A4 AuditRecord (SQLite) . . . 19
VI-B Data Retention and Privacy . . . 19
VI-C Grounding Data . . . 19
VI-C1 CVE Grounding Dataset (`data/cve_grounding.json`) . . . 19
VI-C2 Alias Map (`data/alias_map.json`) . . . 19
VI-D Configuration Data . . . 19

VII Verification and Validation 19
VII-A Test Suite . . . 19
VII-B Evaluation Harness . . . 20
VII-C Empirical Results Summary (900-Record Canonical Runs) . . . 20
VII-C1 Experiment 1 — Raw LLM vs. Full Pipeline . . . 20
VII-C2 Experiment 2 — Model Head-to-Head . . . 20
VII-C3 Experiment 3 — Component Ablation . . . 20
VII-C4 Experiment 4 — Hallucination Deep-Dive (250 Records) . . . 20
VII-D Metric Definitions . . . 20

VIII Deployment 20
VIII-A Docker Deployment (Recommended) . . . 20
VIII-B Manual Deployment . . . 20
VIII-C Configuration Management . . . 21
VIII-C1 Environment Variables (.env) . . . 21
A-0 Feature Flags (config/features.yaml) . . . 21

Appendix A: IntentType Enumeration (9 Values) 21

Appendix B: TargetType Enumeration (6 Values) 21

Appendix C: SubIntentType Enumeration (27 Values) 21

Appendix D: HallucinationClass Enumeration (6 Values) 21

Appendix E: RBAC Policy Table 21

Appendix F: Enforcement Policy Table 22

Appendix G: Adversarial Detection Patterns 22
G-A Layer A: Lexical (15 patterns) 22
G-B Layer B: Structural (7 patterns + 3 heuristics) 22

Appendix H: Configuration Parameters (Complete) 22

Appendix I: Exception Hierarchy 22

Appendix J: Dependency Versions 22

Appendix K: Hashing Algorithms 22

I. Introduction

A. Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the **NeuroShell Intent Recognition Engine (IRE)**, designated Component 01 of the NeuroShell Framework. The IRE is a domain-specific Natural Language Understanding (NLU) gateway that converts natural-language offensive-security operator commands into validated, schema-constrained JSON intent contracts.

This document is intended for:

- **Research evaluators** assessing the IRE's design and empirical performance;
- **Developers** maintaining or extending the pipeline;
- **Downstream consumers** (e.g., Component 02 Dynamic Planner) integrating with the IRE API;
- **Examiners** reviewing the MSc Cybersecurity research project (ref. R26-CS-018).

B. Scope

1) Product Scope: The IRE is a **self-contained FastAPI microservice** that:

1) Accepts natural-language offensive-security commands via a REST API;
2) Applies multi-layer validation (adversarial detection, schema, regex, network architecture, scope);
3) Produces a validated `IntentSchema` or `PlannerContract` JSON output;
4) Enforces role-based access control, session context, and hallucination detection;
5) Maintains a durable, privacy-preserving audit log.

2) In Scope:

- All source code under `src/` (pipeline, middleware, validation, inference, schemas, API, audit, utilities);
- Configuration under `config/` (settings, features, RBAC policy, enforcement policy, LoRA config);
- Grounding data under `data/` (CVE grounding, alias map);
- Evaluation harness under `eval/` (diagnose, metrics, experiment scripts);
- Test suite under `tests/` (unit and integration);
- Deployment artifacts (Dockerfile, docker-compose.yml, scripts).

3) Out of Scope:

- Component 02 (Dynamic Planner), Component 03 (AEERE Executor), and Component 04 (AVAE Vulnerability Analyzer) — these are downstream consumers referenced only for interface-design justification;
- Frontend UI (separate project);
- Network infrastructure and physical security;
- LoRA fine-tuning execution (Phase 2, planned but not yet implemented).

C. Definitions, Acronyms, Abbreviations

TABLE I
Definitions, Acronyms, and Abbreviations.

Term	Definition
IRE	Intent Recognition Engine
NLU	Natural Language Understanding
LLM	Large Language Model
CVE	Common Vulnerabilities and Exposures
RBAC	Role-Based Access Control
CIDR	Classless Inter-Domain Routing
RFC-1918	Address Allocation for Private Internets
SRS	Software Requirements Specification
Tier-1	Single-sample LLM inference path
Tier-2	Conditional N-sample self-consistency path
DAcc	Decision Accuracy
CFAcc	Contract-Field Accuracy
FP	False Positive (incorrectly blocked well-formed input)
S5/S6/S6B/S7	Pipeline validation stages (Schema/Regex/Network/Scope)
M1--M11	Pipeline middleware components
PlannerContract	Machine-readable JSON hand-off object for Component 02
Grounding Hash	SHA-256 digest of CVE and alias datasets for cache invalidation

D. References

References

[1] NeuroShell IRE README. *README.md*, NeuroShell IRE Repository.
[2] IRE Research Extraction. *IRE-Doc/research-extraction.md*.
[3] Experiment Results (900-record). *IRE-Doc/NeuroShell_IRE_Test_New_Results.md*.
[4] Data Collection Plan. *IRE-Doc/Data_Collection_Plan.md*.
[5] Results Section Draft. *IRE-Doc/Results_Section_Draft.md*.
[6] Gaps and Future Works. *IRE-Doc/Gaps & Future Works.md*.
[7] IEEE Standard 830-1998, *Recommended Practice for Software Requirements Specifications*, IEEE Computer Society.
[8] OWASP Top 10 (2021), owasp.org.
[9] RFC 1918 — *Address Allocation for Private Internets*, IETF.
[10] *CVE Format Standard*, mitre.org.

E. Overview

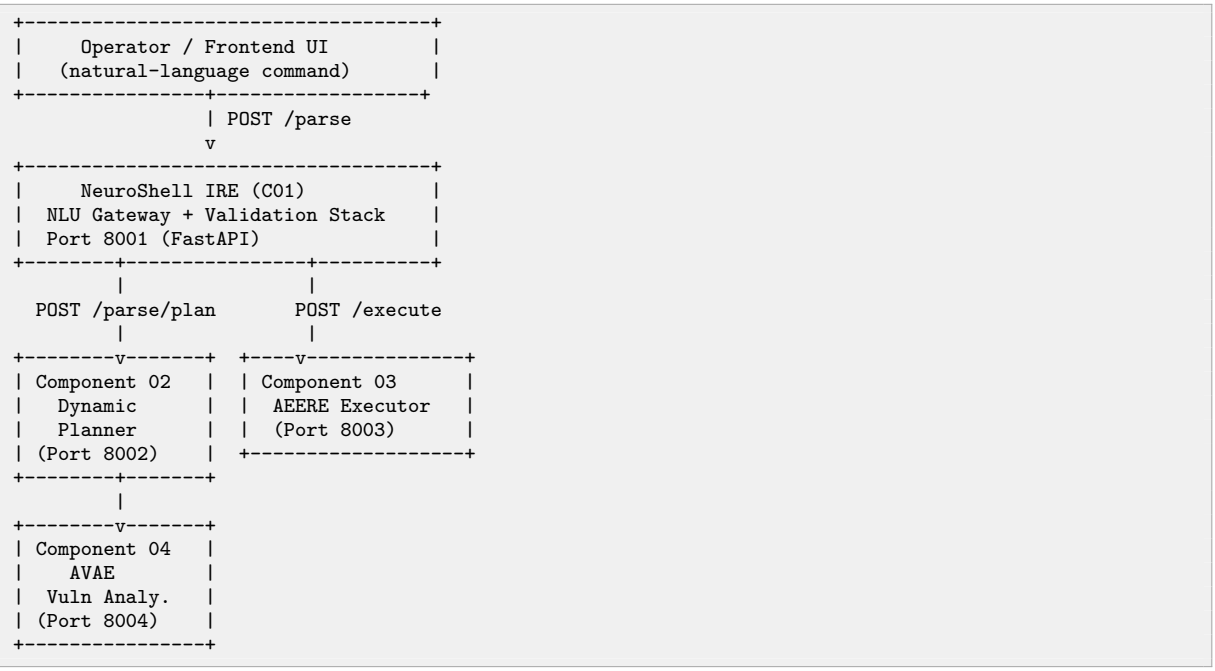
The remainder of this document is organized as follows:

- **Section 2** describes the overall product perspective, user classes, and operating environment;
- **Section 3** specifies all system features and functional requirements in detail;
- **Section 4** defines external interface requirements (API, hardware, software, communication);
- **Section 5** specifies non-functional requirements (performance, security, reliability);
- **Section 6** covers data requirements (models, retention, grounding data);
- **Section 7** describes verification, validation, and empirical results;
- **Section 8** covers deployment procedures;
- **Section 9** provides appendices with enumerations, policy tables, and schema definitions.

II. Overall Description

A. Product Perspective

The NeuroShell IRE is **Component 01** of the NeuroShell Framework, an autonomous penetration testing system. The IRE operates as an **independently deployable microservice** that bridges the gap between human operator intent (natural language) and machine-executable actions (validated JSON contracts).



The IRE does **not** depend on downstream components for core operation. Component 02 (Planner), Component 03 (Executor), and Component 04 (Vulnerability Analyzer) are optional integrations controlled by feature flags.

B. Product Functions

The IRE provides the following core functions:

TABLE II
IRE CORE PRODUCT FUNCTIONS.

Function	Description
Intent Classification	Classify natural-language commands into one of 9 intent types.
Target Extraction	Extract and validate target specification (IP, subnet, domain, URL, hostname).
Entity Extraction	Extract ports, modifiers, CVE IDs, tool hints, and scheduling.
Hallucination Detection	Detect 6 classes of LLM hallucination via structured validation.
Adversarial Detection	Detect prompt injection, jailbreak, and encoded payload attacks.
RBAC Enforcement	Enforce role-based access control (4 roles, 9 intents).
Session Management	Multi-turn context injection with configurable memory window.
Semantic Caching	Exact-hash plus cosine-similarity response cache with grounding invalidation.
Self-Consistency Voting	Tier-2 N-sample consensus with semantic entropy uncertainty quantification.
Contract Generation	Produce `PlannerContract` JSON with tool synthesis and command generation.
Audit Logging	Privacy-preserving (SHA-256 hashed) append-only audit trail.
Escalation Queue	Human-in-the-loop review for high-severity findings with HTTP 202 pattern.

TABLE III
USER CLASSES, PRIVILEGES, AND PERMITTED INTENTS.

Role	Privilege Level	Permitted Intents	Description
viewer	Read-only	None (pre-inference fast-fail)	Observation-only; cannot execute any commands.
analyst	Standard	NETWORK_SCAN, VULNERABILITY_AUDIT, SERVICE_ENUMERATION, PASSIVE_RECON, AMBIGUOUS, REJECTED	Passive and observational operations only.
operator	Elevated	All 9 intents	Active operations including exploitation and password attacks.
admin	Full	All 9 intents	System administration including model switching and policy management.

C. User Classes and Characteristics

Default role: `analyst` (when no role is specified in the request).
Unknown roles: Treated as `viewer` (most restrictive — implicitly denied everywhere).

D. Operating Environment

TABLE IV
HARDWARE REQUIREMENTS.

Component	Minimum	Recommended
CPU	4 cores (2.0 GHz+)	8 cores (3.0 GHz+)
RAM	8 GB	16 GB
GPU	None (CPU-only supported)	NVIDIA CUDA-capable GPU
Storage	10 GB free	50 GB SSD
Network	Loopback (localhost)	LAN (100 Mbps+)

1) Hardware:

TABLE V
SOFTWARE ENVIRONMENT.

Component	Version	Purpose
Python	3.11+	Runtime
Ollama	0.32.x	LLM inference daemon
Docker	24.x (optional)	Containerized deployment
Docker Compose	2.x (optional)	Multi-service orchestration

2) Software:
3) LLM Models:

E. Design and Implementation Constraints

1) **Language:** Python 3.11+ (async-first with FastAPI/uvicorn).
2) **Schema Validation:** Pydantic v2 with `extra="forbid"` (strict schema enforcement).

TABLE VI
LLM MODELS.

Model	Size	Purpose
`gemma4:latest`	9.6 GB	Primary inference (default)
`qwen2.5-coder:7b`	4.7 GB	Alternative inference backbone

3) **Inference Runtime:** Ollama (local daemon, no cloud API calls).
4) **Async Concurrency:** Single-worker uvicorn (pipeline is CPU-bound during LLM inference).
5) **Rate Limiting:** slowapi, 30 requests/minute per IP.
6) **Input Limit:** 2000 characters per command.
7) **Session Limit:** 1024 concurrent sessions, 8 turns per session, 30-minute TTL.
8) **Cache Limits:** 256 entries (inference LRU), 512 entries (semantic cache).
9) **Privacy:** Raw input never stored in plaintext; SHA-256 hashing only.

F. Assumptions and Dependencies

1) Assumptions:

1) Ollama daemon is running and accessible at the configured URL.
2) Required LLM models are pre-pulled and available.
3) The operator has valid credentials for the assigned RBAC role.
4) Network connectivity exists between IRE and the Ollama daemon.
5) The engagement scope CIDR is correctly configured for the target environment.

TABLE VII
EXTERNAL DEPENDENCIES.

Dependency	Type	Purpose
Ollama	Runtime	LLM inference engine
`data/cve_grounding.json`	Data	CVE reference dataset (20 entries)
`data/alias_map.json`	Data	Exploit-name to CVE alias mappings (17 entries)
`all-MiniLM-L6-v2`	ML Model	Sentence embeddings for semantic cache
`en_core_web_sm`	ML Model	spaCy English NLP model

2) Dependencies:

III. SYSTEM FEATURES AND FUNCTIONAL REQUIREMENTS

A. Pipeline Architecture

1) Overview: The IRE implements a **7-stage pipeline with 11 middleware components**, organized as follows:

```
PRE-INFERENCE MIDDLEWARE (M1-M4)
    v
7-STAGE PIPELINE (S1-S7)
    v
POST-INFERENCE MIDDLEWARE (M5-M11)
    v
RESPONSE ASSEMBLY + AUDIT
```

Requirement FR-PIPE-001: The pipeline shall process each request through the full stage sequence (S1→S7) unless an earlier stage raises a blocking exception.

Requirement FR-PIPE-002: Each middleware and stage shall be wrapped in individual try/except blocks so that a failure in non-critical middleware degrades gracefully rather than aborting the request.

Requirement FR-PIPE-003: The pipeline shall accumulate `ValidationFindings` across all stages (S5/S6/S6B/S7) and include them in the response payload regardless of pass/fail status.

Requirement FR-PIPE-004: Every non-passed finding shall be written to the SQLite audit log via `_log_audit_findings`.

2) Stage Sequence:

3) Middleware Sequence:

TABLE VIII
PIPELINE STAGE SEQUENCE (S1–S7).

Stage	Module	Function	Input	Output
S1	`InputNormalizer`	NFKC Unicode normalization, control-char strip, jargon standardization	Raw text	Normalized text
S2	`AliasResolver`	Alias to CVE canonical mapping with provenance annotation	Normalized text	Enriched text
S3	`OllamaInferenceEngine`	Tier-1/Tier-2 LLM inference with self-consistency	Enriched text	`IntentSchema`
S4	`JSONParser`	Strip thought tags, fences; extract JSON object	LLM output text	Parsed dict
S5	`SchemaValidator`	Pydantic v2 `IntentSchema` validation	Dict	`IntentSchema`
S6	`RegexValidator`	IP/CIDR/CVE/port/metachar regex checks	`IntentSchema`	Validated `IntentSchema`
S6B	`NetworkArchitectureValidator`	Type consistency, `ipaddress` parse, scope containment, CIDR sanity	`IntentSchema`	Validated `IntentSchema`
S7	`ScopeGuard`	7 injection regexes on raw input plus RFC-1918 private-range check	Raw input + `IntentSchema`	(`IntentSchema`, warnings)

TABLE IX
MIDDLEWARE SEQUENCE (M1–M11).

Middleware	Module	Phase	Function
M1	`AdversarialDetector`	Pre-inference	Weighted threat scoring → block/allow
M2	`RBACGuard.pre_inference_check`	Pre-inference	Restricted-role fast-fail
M3	`SessionContextStore.inject_context`	Pre-inference	Prepend session turn history
M4	`SemanticCache.lookup`	Pre-inference	Exact-hash then cosine cache hit
M5	`RBACGuard.post_inference_check`	Post-inference	Role × intent authorization
M7	`SemanticCache.store`	Post-inference	Store successful response
M8	`IREResponseV2.from_v1`	Post-inference	Assemble V2 response with metadata
M9	`SubIntentClassifier.enrich`	Post-inference	Hierarchical sub-intent classification
M10	`SessionContextStore.update`	Post-inference	Record completed turn
M11	Session info enrich	Post-inference	Attach turn count to response

B. Pre-Inference Middleware (M1–M4)

1) M1 — AdversarialDetector: **Requirement FR-ADV-001:** The adversarial detector shall scan raw input against 15 lexical injection patterns (weights 0.8–1.0) and 7 structural anomaly patterns (weights 0.5–0.9) plus 3 heuristic checks.

Requirement FR-ADV-002: Any single signal with weight ≥ 0.95 shall trigger immediate blocking (auto-block), regardless of cumulative score.

Requirement FR-ADV-003: The cumulative threat score shall be capped at 1.0, and blocking shall occur when the score meets or exceeds the configured threshold (default 0.70).

Requirement FR-ADV-004: Scanning shall short-circuit as soon as the threshold is reached (efficiency optimization).

Requirement FR-ADV-005: The adversarial detector may be disabled via the `adversarial_detection` feature flag.

Detection categories:

TABLE X
ADVERSARIAL DETECTION LAYERS.

Layer	Patterns	Weight Range	Examples
A: Lexical	15 regex patterns	0.8–1.0	`ignore previous instructions`, `jailbreak`, `<|im_start|>`
B: Structural	7 regex patterns	0.5–0.9	Hex encoding, base64 payloads, `<script>`, `eval()`
B: Heuristic	3 checks	0.3–0.4	Special-char density >30%, token >100 chars, repeated n-grams

Decision rule:

$$\text{BLOCK} \iff \big(\exists\, i \in M(x) :\ w_i \geq 0.95\big) \vee \big(S(x) \geq \tau\big), \tag{1}$$

$$S(x) = \min\Big(1,\ \sum_{i \in M(x)} w_i\Big), \qquad \tau = 0.70. \tag{2}$$

2) M2 — RBACGuard (Pre-Inference): **Requirement FR-RBAC-001:** The pre-inference check shall fast-fail for any role in the `RESTRICTED_ROLES` set (`viewer`).

Requirement FR-RBAC-002: An `RBACError` shall be raised with `intent="ANY"` for restricted roles.

Requirement FR-RBAC-003: The pre-check may be disabled via the `rbac` feature flag.

3) M3 — SessionContextStore: **Requirement FR-SES-001:** The session context store shall maintain per-session turn logs with configurable limits: max 8 turns, max 1024 sessions, 30-minute TTL, LRU eviction.

Requirement FR-SES-002: Context shall be injected in the following format:

```
Session context:
[Turn 1] NETWORK_SCAN on SUBNET "192.168.1.0/24" ports=[22,80]
[Turn 2] VULNERABILITY_AUDIT on IP "10.0.0.5" cves=[CVE-2021-44228]
Current command: <user command>
```

Requirement FR-SES-003: Context injection shall occur BEFORE input normalization and BEFORE cache lookup, so cache keys include session state.

Requirement FR-SES-004: Only successful responses shall be recorded as turns (error responses are not stored).

4) M4 — SemanticCache: **Requirement FR-CACHE-001:** The semantic cache shall support two-tier lookup: exact SHA-256 hash match first, then cosine similarity over all entries.

Requirement FR-CACHE-002: The cosine similarity threshold shall be configurable (default 0.82).

Requirement FR-CACHE-003: Each cache entry shall carry a `grounding_hash` (SHA-256 of `cve_grounding.json` + `alias_map.json`). Entries with stale grounding hashes shall be purged on contact.

Requirement FR-CACHE-004: The cache shall use LRU eviction at capacity (default 512 entries).

Requirement FR-CACHE-005: Only successful responses shall be cached.

Requirement FR-CACHE-006: The embedding model (`all-MiniLM-L6-v2`) shall be lazy-loaded on first use.

C. Core Pipeline Stages (S1–S7)

1) S1 — InputNormalizer: **Requirement FR-NORM-001:** Input normalization shall apply: NFKC Unicode normalization, control-char stripping (ord < 32), think-token removal (`thinking`, `<|think|>`), whitespace collapse.

Requirement FR-NORM-002: A 20-term jargon standardization table shall map offensive-security slang to canonical terms (e.g., `syn scan` → `SYN stealth scan`, `pwn` → `exploit`, `privesc` → `privilege escalation`).

Requirement FR-NORM-003: Empty or whitespace-only input shall raise an `InputValidationError` with an HTTP 200 error envelope.

2) S2 — AliasResolver: **Requirement FR-ALIAS-001:** The alias resolver shall load `data/alias_map.json` (17 active entries) and compile word-boundary patterns sorted longest-first.

Requirement FR-ALIAS-002: Aliases shall be rewritten to `CANONICAL (alias: ORIGINAL)` format so the LLM sees provenance.

Requirement FR-ALIAS-003: Each alias shall be rewritten only at its first occurrence per input.

3) S3 — OllamaInferenceEngine (Tier-1/Tier-2): **Requirement FR-INF-001:** The inference engine shall support two tiers of inference with conditional gating.

Requirement FR-INF-002: Tier-1 shall execute a single Ollama API call at the configured temperature (default 0.1).

Requirement FR-INF-003: The Tier-1 confidence proxy shall be computed as

$$c_{\text{base}} = \begin{cases} o.\text{confidence}, & \text{if parsed to } [0,1], \\ 0.5, & \text{otherwise.} \end{cases} \tag{3}$$

$$c_1 = \begin{cases} \min(c_{\text{base}}, 0.4), & i \in \mathcal{H}, \\ \min(c_{\text{base}}, 0.5), & i \notin \mathcal{H},\ t_v \text{ missing}, \\ c_{\text{base}}, & \text{otherwise}, \end{cases} \tag{4}$$

where i = intent, t_v = target.value, and $\mathcal{H} = \{\text{AMBIGUOUS},\ \text{REJECTED},\ \emptyset\}$.

Requirement FR-INF-004: If $c_1 \geq \gamma$ ($\gamma = 0.7$), the Tier-1 output shall be returned immediately (entropy logged as 0, band "low", tier "1").

Requirement FR-INF-005: Tier-2 shall execute when $c_1 < \gamma$, generating $N = 5$ total samples (Tier-1 reused + $N - 1$ resamples) at a higher temperature (default 0.7).

Requirement FR-INF-006: Tier-2 outputs shall be clustered by semantic-equivalence signature

$$\sigma(o) = \big(\text{intent},\ \text{target_type},\ \text{target_value.lower}(),\ \text{tuple}(\text{sorted}(cve_ids))\big), \tag{5}$$

which deliberately ignores ports and modifiers.

TABLE XI
SCHEMA ERROR TO HALLUCINATION-CLASS MAPPING.

Error Field/Condition	Hallucination Class
ports in error path	FABRICATED_PARAMETER
cve_ids in error path	FABRICATED_CVE
target or intent in error path	TARGET_TYPE_MISMATCH
rejection_reason, confidence, AMBIGUOUS, REJECTED in message	UNGROUNDED_CONFIDENCE
Default fallback	UNGROUNDED_CONFIDENCE

TABLE XII
REGEXVALIDATOR CHECKS.

Check	Description	Failure Class	Severity
1. Target Format	Type-specific regex: IPv4, IPv6, CIDR, Domain, Hostname, URL	TARGET_TYPE_MISMATCH	block
2. CVE Syntax + Grounding	`^CVE-\d{4}-\d{4,7}$` syntax; then lookup in `cve_grounding.json`	FABRICATED_CVE / UNGROUNDED_CONFIDENCE	block / from policy
3. Port Bounds	$\forall p$: $1 \le p \le 65535$	FABRICATED_PARAMETER	block
4. Shell Metacharacters	Scan for `[;&|` $!>]` in `target.value` and modifiers	FABRICATED_PARAMETER	block

Requirement FR-INF-007: Semantic entropy shall be computed as

$$H(x) = -\sum_j p_j \ln p_j, \qquad p_j = |C_j|/N. \tag{6}$$

Requirement FR-INF-008: Entropy bands shall be mapped via configurable thresholds:

- $H < 0.3 \rightarrow$ "low";
- $0.3 \le H \le 0.8 \rightarrow$ "medium";
- $H > 0.8 \rightarrow$ "high".

Requirement FR-INF-009: The resolution method for Tier-2 shall be plurality vote (largest cluster wins; ties resolve by insertion order).

Requirement FR-INF-010: The inference engine shall maintain a two-layer cache: in-memory LRU (256 entries) plus JSON disk persistence, keyed by `SHA-256(model_name|enriched_input)`.

4) S4 — JSONParser: **Requirement FR-JSON-001:** The JSON parser shall strip `thinking...response`, `<|think|>`, and markdown code fences.

Requirement FR-JSON-002: It shall extract the substring from the first `{` to the last `}` (inclusive).

Requirement FR-JSON-003: It shall validate that the result is a Python dict; failure raises `JSONParseError`.

5) S5 — SchemaValidator: **Requirement FR-SCH-001:** The schema validator shall validate the parsed dict against the `IntentSchema` Pydantic v2 model via `model_validate()`.

Requirement FR-SCH-002: On validation failure, a `ValidationFinding` shall be created with `severity="block"` and the appropriate `HallucinationClass` mapped from the failing field:

Requirement FR-SCH-003: Schema constraints include:

- `intent` ∈ 9-class `IntentType` enum;
- `target.type` ∈ 6-class `TargetType` enum;
- `confidence` ∈ [0.0, 1.0];
- `ports[`i`]` ∈ [1, 65535];
- `cve_ids[`i`]` matches `^CVE-\d{4}-\d{4,7}$`;
- `intent=REJECTED` ⇒ `rejection_reason` required;
- `intent=AMBIGUOUS` ⇒ `confidence` < 0.5;
- No extra fields allowed (`extra="forbid"`).

6) S6 — RegexValidator: **Requirement FR-REG-001:** Four sequential checks shall be performed:

Requirement FR-REG-002: CVE syntax failures shall produce severity from `EnforcementPolicy` (default: block).

Requirement FR-REG-003: Syntactically valid but ungrounded CVEs (not in `cve_grounding.json`) shall produce `UNGROUNDED_CONFIDENCE` with severity from policy (default: warn — advisory only).

7) S6B — NetworkArchitectureValidator: **Requirement FR-NET-001:** Four-step validation for IP/SUBNET targets:

Requirement FR-NET-002: Domain, URL, Hostname, and UNKNOWN types shall auto-pass the scope check (scope validation not applicable).

Requirement FR-NET-003: Engagement mode `"disabled"` shall short-circuit all validation; `"warn"` shall downgrade all severities to advisory warnings.

TABLE XIII
NETWORK ARCHITECTURE VALIDATION STEPS.

Step	Check	Failure Class
1. Type Consistency	IP must not contain "/"; SUBNET must contain "/"; UNKNOWN must be empty	TARGET_TYPE_MISMATCH
2. Parseability	`ipaddress.ip_network()` or `ip_address()` parse	format error
3. Scope Containment	IP: addr $\in \Omega$; SUBNET: net $\subseteq \Omega$ (`subnet_of`)	OUT_OF_SCOPE_TARGET
4. CIDR Sanity	Prefix $\geq$ `max_cidr_prefix` (30); reject /32 subnet; IPv6 < /64 rejected	OUT_OF_SCOPE_TARGET

TABLE XIV
INTENTSCHEMA FIELDS.

Field	Type	Required	Constraints
`intent`	IntentType	Yes	9-class enum
`target`	Target	Yes	`{type: TargetType, value: str}`
`ports`	List[int]	No	Each $\in$ [1, 65535]
`modifiers`	List[str]	No	Free text
`cve_ids`	List[str]	No	Each matches `^CVE-\d{4}-\d{4,7}$`
`tool_hint`	Optional[str]	No	Suggested tool
`schedule`	Optional[str]	No	Cron expression
`confidence`	float	Yes	$\in$ [0.0, 1.0]
`rejection_reason`	Optional[str]	Yes if REJECTED	Required when `intent=REJECTED`

8) S7 — ScopeGuard: **Requirement FR-SCOPE-001:** The scope guard shall scan the **raw input** (not the enriched/normalized form) against 7 injection regex patterns. Any match shall produce an unconditional hard block.

Requirement FR-SCOPE-002: Private-range membership shall be checked against 7 network ranges:

- `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`;
- `127.0.0.0/8` (loopback), `169.254.0.0/16` (link-local);
- `::1/128` (IPv6 loopback), `fc00::/7` (IPv6 ULA).

Requirement FR-SCOPE-003: Public (non-private) targets shall trigger `OUT_OF_SCOPE_TARGET` with severity determined by `EnforcementPolicy`:

- `"strict"` mode → block;
- `"research"` mode → warn (advisory).

D. Post-Inference Middleware (M5–M11)

1) M5 — RBACGuard (Post-Inference): **Requirement FR-RBAC-010:** The post-inference check shall verify that the detected intent is permitted for the caller's role using `INTENT_ROLE_POLICY`.

Requirement FR-RBAC-011: Denied intents shall raise `RBACError` listing the detected intent and the role's permitted intents.

2) M7 — SemanticCache (Store): **Requirement FR-CACHE-010:** Successful responses shall be stored with their grounding hash for future invalidation.

3) M8 — IREResponseV2 Assembly: **Requirement FR-RESP-001:** The V2 response shall include all V1 fields plus: `sub_intent`, `secondary_intents`, `uncertainty_band`, `raw_entropy`, `tier_triggered`, `resolution_method`, `cache_hit`, `rbac_role`, `session_turns`, `validation_findings`, `schema_version`.

Requirement FR-RESP-002: Error responses shall follow the 200-envelope pattern: HTTP 200 with `status:"error"`, carrying `error`, `stage`, `detail`, `field`, `validation_findings`, and `latency_ms`.

4) M9 — SubIntentClassifier: **Requirement FR-SUB-001:** The sub-intent classifier shall be a first-match-wins rule table over 27 sub-intent types across 7 parent intents.

Requirement FR-SUB-002: Classification shall be gated: disabled for AMBIGUOUS/REJECTED intents and for confidence < 0.6.

5) M10/M11 — Session Update: **Requirement FR-SES-010:** After successful processing, the session store shall record the turn with intent, target, ports, CVE IDs, and confidence.

E. Schema and Contract System

1) IntentSchema (IRE Internal): The `IntentSchema` is the IRE's internal validated representation:

TABLE XV
PLANNERCONTRACT FIELDS.

Field	Type	Purpose
sub_intent	Optional[SubIntentType]	Hierarchical refinement for C02 command construction
tool_hint	Optional[str]	Tool suggestion (nmap, gobuster, nikto, manual, none)
schedule	Optional[str]	Cron expression; None = execute immediately
scope_warnings	List[str]	SCOPE_WARNING and ARCH_WARNING entries

TABLE XVI
HALLUCINATION CLASSES.

Class	Description	Detection Stage
FABRICATED_CVE	Malformed or non-existent CVE ID	S5 (schema), S6 (regex+grounding)
TARGET_TYPE_MISMATCH	Structural mismatch between target type and value	S5 (schema), S6 (format), S6B (consistency)
CONTRADICTORY_ACTION_TARGET	Incompatible action-target pairing	S5 (schema)
FABRICATED_PARAMETER	Out-of-bounds or injected parameters	S5 (schema), S6 (port/metachar)
UNGROUNDED_CONFIDENCE	Valid syntax but unindexed CVE; low-confidence output	S6 (grounding), S5 (confidence)
OUT_OF_SCOPE_TARGET	Target outside authorized engagement scope	S6B (scope), S7 (RFC-1918)

2) PlannerContract (Component 02 Hand-off): The `PlannerContract` extends the intent schema with downstream-consumption fields:

Executability predicates:

- `is_executable` → False if REJECTED/AMBIGUOUS, or if any `scope_warning` contains "SCOPE_VIOLATION" or "ARCH_WARNING";
- `is_immediate` → True if `schedule` is None;
- `has_scope_issues` → True if any `scope_warnings` present.

F. Hallucination Taxonomy and Enforcement

1) HallucinationClass Enum (6 Classes):

TABLE XVII
DEFAULT ENFORCEMENT BY CLASS.

HallucinationClass	Default Action	Reasoning
FABRICATED_PARAMETER	BLOCK	Execution risk from out-of-bounds parameters or injection
CONTRADICTORY_ACTION_TARGET	BLOCK	Safety invariant violation
TARGET_TYPE_MISMATCH	BLOCK	Structural mismatch prevents downstream execution
FABRICATED_CVE	BLOCK	Complete LLM hallucination
UNGROUNDED_CONFIDENCE	WARN	Novel but plausible CVEs; advisory only
OUT_OF_SCOPE_TARGET	Dynamic	BLOCK in strict mode; WARN in research mode

2) Enforcement Policy:

3) ValidationFinding Schema: Each validation check produces a `ValidationFinding`:

G. Contract Builder and Tool Synthesis

1) Tool Synthesis Table:

2) Priority Ladder:

3) Nmap Flag Mapping:

H. Audit and Escalation System

1) Audit Logger: **Requirement FR-AUDIT-001:** The audit logger shall maintain an append-only SQLite `audit_log` table.

Requirement FR-AUDIT-002: Raw user input shall NEVER be stored in plaintext; only SHA-256 hashes shall be recorded.

Requirement FR-AUDIT-003: Each finding shall be recorded with: timestamp, session_id, raw_input_hash, intent_summary, hallucination_class, severity, enforcement_action, and status.

2) Escalation Queue: **Requirement FR-ESC-001:** Findings with `severity="escalate"` shall be recorded with `status="pending"` in the audit log.

Requirement FR-ESC-002: The `POST /parse/plan` endpoint shall return HTTP 202 with `escalation_id` when an ESCALATE-severity finding is encountered, withholding the contract.

Requirement FR-ESC-003: Admins may approve or reject escalations via `POST /admin/escalations/{id}/resolve`, which releases or discards the cached contract.

TABLE XVIII
VALIDATIONFINDING FIELDS.

Field	Type	Values
validator	str	Which component performed the check
passed	bool	True if check passed
hallucination_class	Optional[HallucinationClass]	Assigned class on failure
detail	str	Human-readable description
severity	Literal[”block”,”warn”,”escalate”]	Enforcement action

TABLE XIX
TOOL SYNTHESIS PER INTENT.

Intent	Tool	Command Template
NETWORK_SCAN	nmap	nmap [flags] <target>
VULNERABILITY_AUDIT	nikto (URL) / nmap	nikto -h <target> [-p port] [-ssl]
SERVICE_ENUMERATION	nmap	nmap -sV [flags] <target>
EXPLOITATION	manual	Requires human authorization
PASSWORD_ATTACK	manual	Lockout risk warning
DIRECTORY_BRUTEFORCE	gobuster	gobuster dir -u <url> -w <wordlist> -q
PASSIVE_RECON	none	Information gathering only
AMBIGUOUS	none	Requires clarification
REJECTED	none	Command rejected

IV. External Interface Requirements

A. User Interfaces

The IRE does not have a direct user interface. It provides a REST API consumed by:

- Frontend UI (separate project, `apps/web/`);
- Downstream components (C02, C03, C04);
- CLI tools and scripts.

B. Hardware Interfaces

C. Software Interfaces

D. Communication Interfaces

1) REST API Base:

- **Protocol:** HTTP/1.1 (HTTPS recommended for production);
- **Content-Type:** `application/json`;
- **Authentication:** `X-API-Key` header (Bearer-style);
- **Rate Limiting:** 30 requests/minute per IP (slowapi);
- **CORS:** Allow all origins (development mode).

2) Response Headers:

E. API Endpoint Specifications

1) GET /: **Purpose:** Service information (no auth required).
Response 200:

```
{
  "service": "NeuroShell IRE",
  "version": "2.0.0",
  "status": "running",
  "docs": "/docs",
  "endpoints": { ... }
}
```

2) GET /health: **Purpose:** System health check (no auth required).
Response 200:

```
{
  "status": "healthy",
  "ollama": { "connected": true, "model": "gemma4:latest" },
  "pipeline": { "stages": 7, "features": { ... } },
  "sessions": { "active": 0, "max": 1024 },
  "semantic_cache": { "size": 0, "max_size": 512 },
```

TABLE XX
CONTRACT PRIORITY LADDER.

Priority	Condition
SCHEDULED	Cron expression is set
CRITICAL	Intent is EXPLOITATION
HIGH	Intent is VULNERABILITY_AUDIT with CVE IDs
MEDIUM	NETWORK_SCAN, SERVICE_ENUMERATION, or VULNERABILITY_AUDIT (no CVEs)
LOW	All other intents

TABLE XXI
MODIFIER TO NMAP-FLAG MAPPING.

Modifier	Flag
stealth	-sS (SYN scan)
aggressive	-A
version_detect	-sV
os_detect	-O
udp	-sU
full_port	-p-
verbose	-v
stealth timing	-T1
Default timing	-T3

```
  "inference_cache": { "size": 0 }
}
```

Response 503: Service unavailable.

3) POST /parse: **Purpose:** Core intent recognition endpoint.

Request:

```
{
  "command": "scan 192.168.1.0/24 for open ports",
  "session_id": "optional-session-id",
  "role": "analyst"
}
```

Constraints:

- `command`: required, non-empty, $\leq$ 2000 characters;
- `role`: optional, default `"analyst"`.

Response 200 (success):

```
{
  "status": "success",
  "intent": "NETWORK_SCAN",
  "target": { "type": "SUBNET", "value": "192.168.1.0/24" },
  "ports": [],
  "modifiers": [],
  "cve_ids": [],
  "tool_hint": null,
  "schedule": null,
  "confidence": 0.85,
  "rejection_reason": null,
  "scope_warnings": [],
  "latency_ms": 27650,
  "sub_intent": "SYN_STEALTH",
  "uncertainty_band": "low",
  "raw_entropy": 0.0,
  "tier_triggered": 1,
  "resolution_method": "single_sample",
  "cache_hit": "none",
  "rbac_role": "analyst",
  "validation_findings": [],
  "schema_version": 2
}
```

Response 200 (error envelope):

```
{
  "status": "error",
  "error": "SCOPE_VIOLATION",
```

TABLE XXII
HARDWARE INTERFACES.

Interface	Specification
Network	TCP/IP over Ethernet or loopback
Ollama Communication	HTTP/1.1 to Ollama daemon (default port 11434)
Storage	Filesystem for SQLite audit log, JSON cache files, grounding data

TABLE XXIII
SOFTWARE INTERFACE MAP.

Interface	Protocol	Target
Ollama API	HTTP POST `/api/generate`	Local Ollama daemon
Component 02	HTTP POST `/plan`	Dynamic Planner (port 8002, optional)
Component 03	HTTP POST	AEERE Executor (port 8003, optional)
Component 04	HTTP POST	AVAE Vulnerability Analyzer (port 8004, optional)
SQLite	File I/O	`data/audit_log.db`
Filesystem	Read	`data/cve_grounding.json`, `data/alias_map.json`

```
  "stage": "scope_guard",
  "detail": "Target outside authorized scope",
  "latency_ms": 150,
  "validation_findings": [...]
}
```

Other Responses: 400 (bad request), 401 (unauthorized), 403 (RBAC denied), 413 (payload too large), 429 (rate limited), 503 (service unavailable).

4) POST /parse/plan: **Purpose:** Intent recognition plus `PlannerContract` generation for Component 02.

Request: Same as `/parse`.

Response 200: `PlannerContract` JSON (see Section 3.5.2).

Response 202: Escalation pending — returns `escalation_id` and `status: "pending_review"`, withholds contract.

Response 422: Contract validation failure.

5) POST /execute: **Purpose:** Full synchronous C1→C2→C3→C4 chain execution.

Request: Same as `/parse`.

Response 200: Combined multi-component result (IRE parse + planner output + executor output + vulnerability analysis).

6) GET /metrics: **Purpose:** Pipeline performance metrics (requires auth).

Response 200:

```
{
  "intent_distribution": { ... },
  "error_counts": { ... },
  "latency_percentiles": { "p50": 16498, "p95": 70893, "p99": ... },
  "entropy_records": { ... },
  "cache_hits": { "exact": 0, "semantic": 0 }
}
```

7) GET/DELETE /session/{session_id}: **Purpose:** Session management (requires auth).

GET Response 200: Session info with turns list.

DELETE Response 200:

```
{ "cleared": true }
```

8) Admin Endpoints:

V. NON-FUNCTIONAL REQUIREMENTS

A. Performance Requirements

1) Latency (Measured on CPU-only Hardware): **Requirement NFR-PERF-001:** Pipeline validation overhead (S5–S7) shall not exceed 5 seconds for single-sample requests.

Requirement NFR-PERF-002: Adversarial detection (M1) shall complete in < 5 ms.

Requirement NFR-PERF-003: The semantic cache lookup shall complete in < 50 ms for up to 512 entries.

Note: All latency figures reflect **CPU-only inference** on an ASUS VivoBook X513EA (Intel i5-1135G7, 4C/8T, 24 GB DDR4). GPU deployment would significantly reduce Tier-1 and Tier-2 latencies.

2) Throughput:

TABLE XXIV
IRE RESPONSE HEADERS.

Header	Description
X-IRE-Schema-Version	Schema version(s) supported: "1,2"
X-IRE-Latency-Ms	Pipeline processing latency in milliseconds
X-IRE-Intent	Detected intent type
X-IRE-Cache-Hit	Cache hit type: "exact", "semantic", or "none"
X-Chain-Cache-Hit	Chain cache hit (on /execute only)

TABLE XXV
ADMINISTRATIVE ENDPOINTS.

Method	Path	Purpose
GET	/admin/rbac/{role}	Role permissions summary
GET	/admin/features	Feature flags + tuning params
POST	/admin/features/reload	Reload features.yaml
GET	/admin/models	List available LLM models + active
POST	/admin/models/switch	Switch active model (persists to .env)
GET	/admin/enforcement-policy	Full enforcement policy
GET	/admin/escalations	Escalation queue
POST	/admin/escalations/{id}/resolve	Approve/reject escalation
GET	/admin/audit/summary	Audit log summary
GET	/admin/chain-cache	Chain cache stats
DELETE	/admin/chain-cache	Clear chain cache
GET	/engagement/scope	Current engagement scope config

B. Safety Requirements

Requirement NFR-SAF-001: The pipeline shall be fail-closed: unknown roles are denied by absence from permit lists; schema violations hard-fail.

Requirement NFR-SAF-002: The pipeline bias shall be toward the safe cell: every observed false positive is an over-block (false rejection), never a dangerous misclassification.

Requirement NFR-SAF-003: Exploitation and password attack intents shall require at least the `operator` role.

Requirement NFR-SAF-004: Shell metacharacters in target/modifiers shall be blocked regardless of RBAC role.

Requirement NFR-SAF-005: The pipeline shall produce ESCALATE findings (not ALLOW) when enforcement severity is "escalate", deferring to human review.

C. Security Requirements

1) Input Security: **Requirement NFR-SEC-001:** The pipeline shall defend against 4 adversary capabilities:

1) **T1 — Prompt injection:** Lexical injection patterns, structural anomaly detection, LLM self-rejection.
2) **T2 — Privilege abuse:** Pre-inference role fast-fail, post-inference role×intent policy.
3) **T3 — Model hallucination:** Schema/regex/network/scope validation stack.
4) **T4 — Out-of-scope targets:** CIDR containment, RFC-1918 check, scope mode enforcement.

2) Data Security: **Requirement NFR-SEC-002:** Raw user input shall NEVER be stored in plaintext. Only SHA-256 hashes shall be recorded in the audit log.

Requirement NFR-SEC-003: API authentication shall use the `X-API-Key` header. The default development key (`dev_insecure_key`) shall NOT be used in production.

Requirement NFR-SEC-004: CORS shall be restricted to specific origins in production (current default: allow all, development mode only).

3) Injection Defense Layers:

D. Reliability and Availability

Requirement NFR-REL-001: Non-critical middleware failures (session context, adversarial detection, cache store) shall degrade gracefully without aborting the pipeline.

Requirement NFR-REL-002: The audit logger shall be append-only with no delete operations.

Requirement NFR-REL-003: The semantic cache shall self-heal via grounding-hash invalidation when reference datasets change.

Requirement NFR-REL-004: Docker health checks shall verify IRE (30s interval) and Ollama (15s interval) connectivity.

TABLE XXVI
LATENCY METRICS.

Metric	Raw LLM	Full Pipeline	Notes
p50	11.5–13.5 s	16.5 s	Pipeline adds ~3–5 s validation overhead
p95	16.5–63.3 s	70.9 s	Dominated by Tier-2 spikes on uncertain inputs
p99	17.4–93.7 s	~150 s	Rare Tier-2 chains
Mean	13.4–25.3 s	28.8 s	Weighted by Tier-1 majority

TABLE XXVII
THROUGHPUT LIMITS.

Metric	Value
Rate limit	30 requests/minute per IP
Concurrent sessions	1024
Cache capacity	256 (inference) + 512 (semantic)

E. Maintainability

Requirement NFR-MNT-001: All pipeline stages and middleware shall be individually configurable via feature flags in `config/features.yaml`.

Requirement NFR-MNT-002: Enforcement policy shall be environment-configurable (research/strict/production) without code changes.

Requirement NFR-MNT-003: The test suite shall achieve ≥ 80% line coverage (current: 434 unit + 48 integration = 482 tests).

F. Portability

Requirement NFR-PORT-001: The IRE shall run on Linux, macOS, and Windows with Python 3.11+.

Requirement NFR-PORT-002: Docker deployment shall use the `python:3.11-slim` base image.

Requirement NFR-PORT-003: All configuration shall be environment-variable-driven via the `.env` file or Docker environment.

VI. DATA REQUIREMENTS

A. Logical Data Model

1) Core Entities:

```
IntentSchema
+-- intent: IntentType (9 values)
+-- target: Target {type: TargetType, value: str}
+-- ports: List[int]
+-- modifiers: List[str]
+-- cve_ids: List[str]
+-- tool_hint: Optional[str]
+-- schedule: Optional[str]
+-- confidence: float [0.0, 1.0]
+-- rejection_reason: Optional[str]

PlannerContract (extends IntentSchema)
+-- sub_intent: Optional[SubIntentType]
+-- scope_warnings: List[str]
+-- executability predicates

IREResponseV2 (extends IntentSchema)
+-- validation_findings: List[ValidationFinding]
+-- uncertainty metadata
+-- session context
+-- XAI metadata (optional)
```

2) ValidationFinding:

```
ValidationFinding
+-- validator: str
+-- passed: bool
+-- hallucination_class: Optional[HallucinationClass]
+-- detail: str
+-- severity: "block" | "warn" | "escalate"
```

TABLE XXVIII
INJECTION DEFENSE LAYERS.

Layer	Target	Mechanism
M1 AdversarialDetector	Raw input	15 lexical + 7 structural + 3 heuristic patterns
S7 ScopeGuard	Raw input	7 injection regexes (independent of M1)
S4 JSONParser	LLM output	Strip thought tags, fences
S5 SchemaValidator	Parsed JSON	Pydantic `extra="forbid"` (no extra fields)
S6 RegexValidator	Validated schema	Shell metacharacter scan

TABLE XXIX
DATA TYPES, RETENTION, AND PRIVACY.

Data Type	Retention	Privacy
Raw user input	Transient (in-memory only)	Never stored
Audit log	Append-only, persistent	SHA-256 hashed input only
Semantic cache	LRU eviction (512 max)	In-memory + optional disk
Inference cache	LRU eviction (256 max)	Disk: `eval/.ollama_inference_cache.json`
Session context	30-minute TTL, LRU eviction	In-memory only, never persisted

3) CacheEntry (Semantic Cache):

```
CacheEntry
+-- key_hash: str (SHA-256)
+-- embedding: List[float] (384-dim)
+-- response: IREResponse
+-- grounding_hash: str (SHA-256)
+-- hit_count: int
+-- created_at: float
+-- last_hit_at: float
```

4) AuditRecord (SQLite):

```
audit_log
+-- id: INTEGER PRIMARY KEY
+-- timestamp: TEXT (ISO 8601)
+-- session_id: TEXT (nullable)
+-- raw_input_hash: TEXT (SHA-256)
+-- intent_summary: TEXT
+-- hallucination_class: TEXT
+-- severity: TEXT
+-- enforcement_action: TEXT
+-- status: TEXT ("recorded" | "pending" | "approved" | "rejected")
+-- resolved_by: TEXT
+-- resolved_at: TEXT
+-- resolution_note: TEXT
+-- cached_contract_json: TEXT
```

B. Data Retention and Privacy

C. Grounding Data

1) CVE Grounding Dataset (`data/cve_grounding.json`):

- **Purpose:** Reference dataset for CVE validation (Check 2 of RegexValidator);
- **Format:** JSON with 20 curated entries plus a `_meta` block;
- **Schema per entry:** {`id, description, affected_service, severity`};
- **Refresh:** Manual via `scripts/update_cvegrounding.py` (CISA KEV catalog);
- **Grounding Hash:** SHA-256 digest computed on each cache lookup; stale entries purged.

2) Alias Map (`data/alias_map.json`):

- **Purpose:** Map exploit names and slang to canonical CVE identifiers;
- **Entries:** 10 exploit-name to CVE mappings + 8 slang/term normalizations;
- **Format:** JSON object {`alias: canonical, ...`};
- **Processing:** Word-boundary regex compilation, longest-first matching.

D. Configuration Data

VII. VERIFICATION AND VALIDATION

A. Test Suite

Key behavioral assertions:

TABLE XXX
CONFIGURATION FILES.

File	Format	Purpose
config/settings.py	pydantic-settings	Environment-driven configuration (30+ fields)
config/features.yaml	YAML	Feature flags (10) + tuning parameters (11)
config/rbac_policy.py	Python	Role to intent permission table
config/enforcement_policy.py	Python	HallucinationClass to action mapping
config/lora_config.yaml	YAML	Phase-2 fine-tuning plan

TABLE XXXI
TEST SUITE SUMMARY.

Category	Count	Files	Notable Coverage
Unit tests	434	22 files under tests/unit/	adversarial (42), sub_intent (46), regex (40), semantic_cache (37), session_context (35)
Integration tests	48	5 files under tests/integration/	API v1 (12), API v2 (33), audit escalation (3)
Total	**482**	**27 files**	

- Tier-2 gating calls the LLM exactly once when confident (test_self_consistency_entropy.py);
- Near-duplicate cache isolation and grounding invalidation (test_semantic_cache_adversarial.py);
- IPv6 and production-mode edge coverage (test_uncovered_source_behaviors.py).

B. Evaluation Harness

C. Empirical Results Summary (900-Record Canonical Runs)

1) Experiment 1 — Raw LLM vs. Full Pipeline: **Pipeline lift:** +8pp/+13pp accuracy, +72pp/+64pp catch, −18pp/−16pp FP.

2) Experiment 2 — Model Head-to-Head:

3) Experiment 3 — Component Ablation: **Component contribution ranking (by accuracy impact):**

1) ScopeGuard (+9.5pp);
2) Entire Validation Stack (+9.5pp);
3) SchemaValidator + RegexValidator (+8.1pp);
4) Semantic Cache (+7.5pp);
5) Tier-2 Self-Consistency (+6.0pp);
6) NetworkArchitectureValidator (+5.5pp);
7) RBACGuard (−1.0pp).

4) Experiment 4 — Hallucination Deep-Dive (250 Records): **Largest structural gap:** llm_no_scope_detection (35% of all misses) — public/out-of-scope targets reach the LLM before any scope gate.

D. Metric Definitions

VIII. DEPLOYMENT

A. Docker Deployment (Recommended)

```
# 1. Clone and configure
cd apps/services/neuroshell-ire
cp .env.example .env
# Edit .env: set IRE_API_KEY, OLLAMA_MODEL

# 2. Build and start
docker-compose up -d --build

# 3. Verify
curl http://localhost:8001/health
```

Services:

Volumes: ollama_models mounted at /root/.ollama (persists downloaded models).

B. Manual Deployment

TABLE XXXII
EVALUATION HARNESS SCRIPTS.

Script	Purpose	Path
diagnose.py	Full-pipeline runner → JSONL debug rows + per-category report	eval/diagnose.py
metrics.py	Decision/accuracy/catch/cache metrics	eval/metrics.py
category_metrics.py	NormalizedRow-based metrics engine (canonical)	eval/category_metrics.py
run_baseline.py	Raw unconstrained LLM baseline scorer	eval/run_baseline.py
exp1_raw_vs_pipeline.py	Exp1: raw vs. pipeline, 4 configs	eval/Data Collection/
exp2_model_head_to_head.py	Exp2: Gemma vs. Qwen	eval/Data Collection/
exp3_ablation_study.py	Exp3: 8-config × 900-record ablation	eval/Data Collection/
exp4_hallucination_analysis.py	Exp4: per-class deep-dive (250 records)	eval/Data Collection/

TABLE XXXIII
EXPERIMENT 1 RESULTS.

Metric	Raw Gemma	Raw Qwen	Pipeline Gemma	Pipeline Qwen
Intent Accuracy	78.0%	69.0%	**86.0%**	**82.0%**
Hallucination Catch	0.0%	0.0%	**72.0%**	**64.0%**
False Positive Rate	32.0%	31.0%	**14.0%**	**15.0%**

```
# 1. Prerequisites
python --version  # 3.11+
ollama --version  # 0.32.x

# 2. Virtual environment
python -m venv .venv
.venv\Scripts\activate  # Windows
# source .venv/bin/activate  # Linux/macOS

# 3. Dependencies
pip install -r requirements.txt
python -m spacy download en_core_web_sm

# 4. Configuration
cp .env.example .env
# Edit .env: set IRE_API_KEY, OLLAMA_MODEL, OLLAMA_BASE_URL

# 5. Pull LLM model
ollama pull gemma4:e4b

# 6. Start server
python scripts/run_dev.py
```

C. Configuration Management

1) Environment Variables (.env):

2) Feature Flags (config/features.yaml):

APPENDIX A
INTENTTYPE ENUMERATION (9 VALUES)

APPENDIX B
TARGETTYPE ENUMERATION (6 VALUES)

APPENDIX C
SUBINTENTTYPE ENUMERATION (27 VALUES)

APPENDIX D
HALLUCINATIONCLASS ENUMERATION (6 VALUES)

APPENDIX E
RBAC POLICY TABLE

Sensitive intents (require operator+): EXPLOITATION, PASSWORD_ATTACK, DIRECTORY_BRUTEFORCE.

TABLE XXXIV
EXPERIMENT 2 RESULTS.

Metric	Gemma 4	Qwen 2.5 Coder	Delta
Intent Accuracy	86.0%	82.0%	−4.0pp
Hallucination Catch	72.0%	64.0%	−8.0pp
FP Rate	14.0%	15.0%	+1.0pp
p50 Latency	27,650 ms	32,755 ms	+18%

TABLE XXXV
EXPERIMENT 3 ABLATION RESULTS.

Config	DAcc	Catch	FP	CFAcc	Mean Latency
Full Pipeline	**73.5%**	**84%**	34%	87.5%	28,805 ms
No Scope Guard	64.0%	73%	44%	83.9%	31,404 ms
No RBAC Guard	74.5%	64%	22%	88.2%	28,260 ms
No Network Validator	68.0%	82%	44%	83.9%	27,450 ms
No Hallucination Validator	65.4%	62%	44%	82.1%	26,722 ms
Validation Stack Disabled	64.0%	41%	13%	87.4%	25,198 ms
Self-Consistency Disabled	67.5%	80%	45%	85.5%	11,809 ms
Semantic Cache Disabled	66.0%	79%	47%	86.8%	22,949 ms

APPENDIX F
ENFORCEMENT POLICY TABLE

APPENDIX G
ADVERSARIAL DETECTION PATTERNS

A. Layer A: Lexical (15 patterns)

B. Layer B: Structural (7 patterns + 3 heuristics)

APPENDIX H
CONFIGURATION PARAMETERS (COMPLETE)

APPENDIX I
EXCEPTION HIERARCHY

```
IREError (base)
+-- JSONParseError          (stage: "json_parser")
+-- SchemaValidationError   (stage: "schema_validator")
+-- RegexValidationError    (stage: "regex_validator")
+-- NetworkValidationError  (stage: "network_validator")
|   +-- extra: validation_type
+-- ScopeError              (stage: "scope_guard")
+-- InferenceError          (stage: "inference")
+-- RBACError               (stage: "rbac_guard")
    +-- extra: role, denied_intent
```

APPENDIX J
DEPENDENCY VERSIONS

APPENDIX K
HASHING ALGORITHMS

— End of Document —

TABLE XXXVI
EXPERIMENT 4 PER-CATEGORY CATCH RATES.

Category	Total	Caught	Catch %
rbac_violation	12	12	**100.0%**
adversarial_injection	45	43	**95.6%**
fabricated_parameter	32	28	87.5%
target_type_mismatch	32	26	81.2%
fabricated_cve	32	25	78.1%
contradictory_action_target	32	19	59.4%
out_of_scope	45	21	**46.7%**
ambiguous	20	8	40.0%
Total	**250**	**182**	**72.8%**

TABLE XXXVII
METRIC DEFINITIONS.

Metric	Formula
DAcc (Decision Accuracy)	`sum/N_total` of correct decisions: well_formed→ALLOWED+correct fields, flawed→BLOCKED/ESCALATED
Catch (Flawed-Input Catch)	`|flawed records with BLOCKED/ESCALATED|/N_flawed`
FP (False Positive Rate)	`|well_formed records with non-ALLOWED decision|/N_well_formed`
CFAcc (Contract-Field Accuracy)	`|ALLOWED records with correct intent+target|/|ALLOWED well_formed|`

TABLE XXXVIII
DOCKER SERVICES.

Service	Container	Port	Health Check
ire-service	neuroshell-ire	8001	`curl -f http://localhost:8001/health`
ollama	neuroshell-ollama	11434	`curl -f http://localhost:11434/api/tags`

TABLE XXXIX
ENVIRONMENT VARIABLES.

Variable	Default	Description
IRE_API_KEY	dev_insecure_key	API authentication key
OLLAMA_BASE_URL	http://localhost:11434	Ollama daemon URL
OLLAMA_MODEL	gemma4:e4b	Active LLM model
SCOPE_MODE	research	Scope enforcement mode
ENGAGEMENT_SCOPE	192.168.0.0/16	Authorised target CIDR
LOG_LEVEL	INFO	Logging level
PORT	8001	Server port

TABLE XL
FEATURE FLAGS.

Flag	Default	Description
session_context	true	Multi-turn session memory
rbac	true	Role-based access control
adversarial_detection	true	Adversarial input detection
semantic_cache	true	Embedding-based response cache
sub_intent	true	Hierarchical sub-intent classification
schema_v2	true	V2 schema routing
xai	false	XAI token attribution
uncertainty_estimation	false	Monte Carlo dropout
continual_learning	false	EWC active learning
speculative_decoding	false	Draft model acceleration

TABLE XLI
INTENT TYPES.

Value	Category	Description
NETWORK_SCAN	Active Recon	Port scanning, host discovery, OS detection
VULNERABILITY_AUDIT	Assessment	CVE-specific or service-specific vulnerability checks
DIRECTORY_BRUTEFORCE	Enumeration	Web directory/file brute-forcing
SERVICE_ENUMERATION	Enumeration	Service version detection, banner grabbing
EXPLOITATION	Attack	Remote code execution, privilege escalation
PASSWORD_ATTACK	Attack	Brute force, credential stuffing, password spray
PASSIVE_RECON	Recon	DNS lookup, WHOIS, OSINT gathering
AMBIGUOUS	Special	Confident < 0.5; requires clarification
REJECTED	Special	Command rejected (injection, policy, etc.)

TABLE XLII
TARGET TYPES.

Value	Format	Example
IP	IPv4 address	192.168.1.1
SUBNET	CIDR notation	192.168.1.0/24
DOMAIN	Domain name	example.com
URL	HTTP/HTTPS URL	https://example.com/api
HOSTNAME	Single-label hostname	webserver01
UNKNOWN	Empty/unspecified	(empty string)

TABLE XLIII
SUB-INTENT TYPES BY PARENT INTENT.

Parent Intent	Sub-Intents
NETWORK_SCAN	ICMP_DISCOVERY, SYN_STEALTH, UDP_SWEEP, VERSION_DETECT, OS_DETECT, FULL_PORT
EXPLOITATION	RCE, PRIV_ESC, LATERAL, PERSISTENCE
VULNERABILITY_AUDIT	CVE_SPECIFIC, SERVICE_SPECIFIC, FULL_SCAN
SERVICE_ENUMERATION	SMB, SNMP, VERSION_DETECT, BANNER_GRAB
PASSWORD_ATTACK	SPRAY, CREDENTIAL_STUFFING, BRUTE_FORCE
PASSIVE_RECON	DNS, WHOIS, OSINT
DIRECTORY_BRUTEFORCE	API, FILES, WEB

TABLE XLIV
HALLUCINATION CLASSES AND ENFORCEMENT.

Class	Detection	Default Enforcement
FABRICATED_CVE	S5 + S6 (syntax)	BLOCK
TARGET_TYPE_MISMATCH	S5 + S6 (format) + S6B	BLOCK
CONTRADICTORY_ACTION_TARGET	S5 (schema)	BLOCK
FABRICATED_PARAMETER	S5 + S6 (port/metachar)	BLOCK
UNGROUNDED_CONFIDENCE	S5 (confidence) + S6 (grounding)	WARN
OUT_OF_SCOPE_TARGET	S6B (scope) + S7 (RFC-1918)	Dynamic (BLOCK/WARN)

TABLE XLV
RBAC POLICY MATRIX.

Intent	viewer	analyst	operator	admin
NETWORK_SCAN	—	✓	✓	✓
VULNERABILITY_AUDIT	—	✓	✓	✓
DIRECTORY_BRUTEFORCE	—	—	✓	✓
SERVICE_ENUMERATION	—	✓	✓	✓
EXPLOITATION	—	—	✓	✓
PASSWORD_ATTACK	—	—	✓	✓
PASSIVE_RECON	—	✓	✓	✓
AMBIGUOUS	—	✓	✓	✓
REJECTED	—	✓	✓	✓

TABLE XLVI
ENFORCEMENT POLICY BY ENVIRONMENT MODE.

HallucinationClass	research	strict	production
FABRICATED_PARAMETER	BLOCK	BLOCK	BLOCK
CONTRADICTORY_ACTION_TARGET	BLOCK	BLOCK	BLOCK
TARGET_TYPE_MISMATCH	BLOCK	BLOCK	BLOCK
FABRICATED_CVE	BLOCK	BLOCK	BLOCK
UNGROUNDED_CONFIDENCE	WARN	WARN	WARN
OUT_OF_SCOPE_TARGET	WARN	BLOCK	BLOCK

TABLE XLVII
LAYER A LEXICAL PATTERNS.

Pattern	Weight
ignore (all)?(previous|prior|above) instructions?	0.9
forget (everything|all|your instructions?)	0.9
you are now (?!scanning|checking|running)	0.8
act as (a)?(different|new|another|evil|unrestricted)	0.8
jailbreak	1.0
dan mode	1.0
prompt injection	0.9
(system|assistant|user):\s*you (must|shall|will)	0.85
<|im_(start|end)|>	1.0
<|think|>	0.95
[INST] or [/INST]	0.9
### (instruction|system|human|assistant)	0.85
disregard (all)?(prior|previous|your)	0.85
new instructions:	0.8
override (previous)?(instructions?|rules?|guidelines?)	0.85

TABLE XLVIII
LAYER B STRUCTURAL PATTERNS.

Pattern	Weight
(\x[0-9a-f]{2}){4,}	0.6
(%[0-9a-f]{2}){4,}	0.5
base64:\s*[A-Za-z0-9+/]{20,}	0.7
<script[\s>]	0.8
\${[^]}{0,50}\}	0.6
eval(|exec(|__import__()	0.9
rm -rf|del /[sq]|format c:	0.8

TABLE XLIX
LAYER B HEURISTICS.

Heuristic	Threshold	Weight
Special-char density	> 30% (for inputs > 20 chars)	0.4
Single token length	> 100 chars	0.3
Repeated n-gram	> 3 occurrences of 3/4/5-grams (≥ 10 chars)	0.35

TABLE L
ALL CONFIGURATION PARAMETERS.

Parameter	Default	Type	Source
ire_api_key	dev_insecure_key	str	.env
port	8001	int	.env
ollama_base_url	http://localhost:11434	str	.env
ollama_model	gemma4:e4b	str	.env
ollama_timeout_seconds	60	int	.env
inference_temperature	0.1	float	.env
inference_max_tokens	2048	int	.env
self_consistency_trigger_threshold	0.7	float	.env
self_consistency_samples	5	int	.env
self_consistency_temperature	0.7	float	.env
entropy_low_threshold	0.3	float	.env
entropy_high_threshold	0.8	float	.env
scope_mode	research	literal	.env
engagement_scope	192.168.0.0/16	str	.env
engagement_name	Lab Environment	str	.env
engagement_mode	warn	literal	.env
max_cidr_prefix	30	int	.env
min_cidr_prefix	32	int	.env
lru_cache_max_size	256	int	.env
planner_enabled	false	bool	.env
planner_url	http://127.0.0.1:8002	str	.env
planner_api_key	neuroshell-c2-secret-key	str	.env
planner_timeout_seconds	300	int	.env
component3_enabled	false	bool	.env
component3_url	http://127.0.0.1:8003	str	.env
component4_enabled	false	bool	.env
component4_url	http://127.0.0.1:8004	str	.env
session_ttl_minutes	30	int	features.yaml
session_max_turns	8	int	features.yaml
session_max_sessions	1024	int	features.yaml
semantic_cache_threshold	0.82	float	features.yaml
semantic_cache_max_size	512	int	features.yaml
adversarial_classifier_threshold	0.7	float	features.yaml
sub_intent_confidence_threshold	0.6	float	features.yaml

TABLE LI
PACKAGE DEPENDENCY VERSIONS.

Package	Version	Purpose
fastapi	0.115.0	Async REST framework
uvicorn[standard]	0.30.6	ASGI server
pydantic	≥2.9, <3.0	Schema validation
pydantic-settings	2.4.0	Environment config
jsonschema	4.23.0	JSON Schema support
spacy	3.7.6	NLP preprocessing
ollama	0.3.3	LLM client
httpx	0.27.2	HTTP client
structlog	24.4.0	Structured logging
sentence-transformers	3.0.1	Semantic cache embeddings
slowapi	0.1.9	Rate limiting
pytest	8.3.2	Test framework
pytest-asyncio	0.23.8	Async test support

TABLE LII
HASH ALGORITHM USES.

Use	Algorithm	Input
Audit input privacy	SHA-256	Raw user input
Semantic cache exact key	SHA-256	Lowercased, stripped input text
Inference cache key	SHA-256	model_name|enriched_input
Grounding invalidation	SHA-256	Concatenation of cve_grounding.json + alias_map.json